合肥大學
HEFEI UNIVERSITY

Programming with Python

27. Funktionen in Modulen

Thomas Weise (汤卫思)
tweise@hfuu.edu.cn

Institute of Applied Optimization (IAO) | 应用优化研究所
School of Artificial Intelligence and Big Data | 人工智能与大数据学院
Hefei University | 合肥大学
Hefei, Anhui, China | 中国安徽省合肥市

Version: 2025-09-05

Programming with Python

Dies ist ein Kurs über das Programmieren mit der Programmiersprache Python an der Universität Hefei (合肥大学).

Die Webseite mit dem Lehrmaterial dieses Kurses ist https://thomasweise.github.io/programmingWithPython (siehe auch den QR-Kode unten rechts). Dort können Sie das Kursbuch (in Englisch) und diese Slides finden. Das Repository mit den Beispielprogrammen in Python finden Sie unter https://github.com/thomasWeise/programmingWithPythonCode.

Outline

1. Einleitung
2. Beispiel
3. Zusammenfassung

Einleitung

Einleitung

- Es ist Ihnen vielleicht nicht aufgefallen, aber wir haben gerade wieder einen großen Schritt in unseren Programmierfähigkeiten gemacht.

Einleitung

- Es ist Ihnen vielleicht nicht aufgefallen, aber wir haben gerade wieder einen großen Schritt in unseren Programmierfähigkeiten gemacht.
- Wir sind von einzelnen Programmen, die nur aus einem Block von Kode bestehen, zu modularen Programmen fortgeschritten.

Einleitung

- Es ist Ihnen vielleicht nicht aufgefallen, aber wir haben gerade wieder einen großen Schritt in unseren Programmierfähigkeiten gemacht.
- Wir sind von einzelnen Programmen, die nur aus einem Block von Kode bestehen, zu modularen Programmen fortgeschritten.
- Wir können nun Kode wiederverwenden.

Einleitung

- Es ist Ihnen vielleicht nicht aufgefallen, aber wir haben gerade wieder einen großen Schritt in unseren Programmierfähigkeiten gemacht.
- Wir sind von einzelnen Programmen, die nur aus einem Block von Kode bestehen, zu modularen Programmen fortgeschritten.
- Wir können nun Kode wiederverwenden.
- Als wir unsere Reise begonnen haben, haben wir unsere Befehle einen nach dem anderen in den Python-Interpreter eingetippt und ausgeführt.

Einleitung

- Es ist Ihnen vielleicht nicht aufgefallen, aber wir haben gerade wieder einen großen Schritt in unseren Programmierfähigkeiten gemacht.
- Wir sind von einzelnen Programmen, die nur aus einem Block von Kode bestehen, zu modularen Programmen fortgeschritten.
- Wir können nun Kode wiederverwenden.
- Als wir unsere Reise begonnen haben, haben wir unsere Befehle einen nach dem anderen in den Python-Interpreter eingetippt und ausgeführt.
- Dann haben wir den Kode in Dateien geschrieben, wodurch wir dann unsere Programme mehrmals ausführen konnten.

Einleitung

- Es ist Ihnen vielleicht nicht aufgefallen, aber wir haben gerade wieder einen großen Schritt in unseren Programmierfähigkeiten gemacht.
- Wir sind von einzelnen Programmen, die nur aus einem Block von Kode bestehen, zu modularen Programmen fortgeschritten.
- Wir können nun Kode wiederverwenden.
- Als wir unsere Reise begonnen haben, haben wir unsere Befehle einen nach dem anderen in den Python-Interpreter eingetippt und ausgeführt.
- Dann haben wir den Kode in Dateien geschrieben, wodurch wir dann unsere Programme mehrmals ausführen konnten.
- Jetzt können wir unseren Kode in Funktionen strukturieren, die wir mit Docstrings und Type Hints erklären können.

Einleitung

- Es ist Ihnen vielleicht nicht aufgefallen, aber wir haben gerade wieder einen großen Schritt in unseren Programmierfähigkeiten gemacht.
- Wir sind von einzelnen Programmen, die nur aus einem Block von Kode bestehen, zu modularen Programmen fortgeschritten.
- Wir können nun Kode wiederverwenden.
- Als wir unsere Reise begonnen haben, haben wir unsere Befehle einen nach dem anderen in den Python-Interpreter eingetippt und ausgeführt.
- Dann haben wir den Kode in Dateien geschrieben, wodurch wir dann unsere Programme mehrmals ausführen konnten.
- Jetzt können wir unseren Kode in Funktionen strukturieren, die wir mit Docstrings und Type Hints erklären können.
- Jetzt können wir schon nützliche und vernünftig große Programme schreiben.

Einleitung

- Es ist Ihnen vielleicht nicht aufgefallen, aber wir haben gerade wieder einen großen Schritt in unseren Programmierfähigkeiten gemacht.
- Wir sind von einzelnen Programmen, die nur aus einem Block von Kode bestehen, zu modularen Programmen fortgeschritten.
- Wir können nun Kode wiederverwenden.
- Als wir unsere Reise begonnen haben, haben wir unsere Befehle einen nach dem anderen in den Python-Interpreter eingetippt und ausgeführt.
- Dann haben wir den Kode in Dateien geschrieben, wodurch wir dann unsere Programme mehrmals ausführen konnten.
- Jetzt können wir unseren Kode in Funktionen strukturieren, die wir mit Docstrings und Type Hints erklären können.
- Jetzt können wir schon nützliche und vernünftig große Programme schreiben.
- Aber bis jetzt sind unsere gesamten Programme noch in einzelnen Datei gespeichert.

Einleitung

- Es ist Ihnen vielleicht nicht aufgefallen, aber wir haben gerade wieder einen großen Schritt in unseren Programmierfähigkeiten gemacht.
- Wir sind von einzelnen Programmen, die nur aus einem Block von Kode bestehen, zu modularen Programmen fortgeschritten.
- Wir können nun Kode wiederverwenden.
- Als wir unsere Reise begonnen haben, haben wir unsere Befehle einen nach dem anderen in den Python-Interpreter eingetippt und ausgeführt.
- Dann haben wir den Kode in Dateien geschrieben, wodurch wir dann unsere Programme mehrmals ausführen konnten.
- Jetzt können wir unseren Kode in Funktionen strukturieren, die wir mit Docstrings und Type Hints erklären können.
- Jetzt können wir schon nützliche und vernünftig große Programme schreiben.
- Aber bis jetzt sind unsere gesamten Programme noch in einzelnen Datei gespeichert.
- Das bringt uns an eine gewisse Grenze, für die Komplexität von den Applikationen, die wir bauen können.

Einleitung

- Es ist Ihnen vielleicht nicht aufgefallen, aber wir haben gerade wieder einen großen Schritt in unseren Programmierfähigkeiten gemacht.
- Wir sind von einzelnen Programmen, die nur aus einem Block von Kode bestehen, zu modularen Programmen fortgeschritten.
- Wir können nun Kode wiederverwenden.
- Als wir unsere Reise begonnen haben, haben wir unsere Befehle einen nach dem anderen in den Python-Interpreter eingetippt und ausgeführt.
- Dann haben wir den Kode in Dateien geschrieben, wodurch wir dann unsere Programme mehrmals ausführen konnten.
- Jetzt können wir unseren Kode in Funktionen strukturieren, die wir mit Docstrings und Type Hints erklären können.
- Jetzt können wir schon nützliche und vernünftig große Programme schreiben.
- Aber bis jetzt sind unsere gesamten Programme noch in einzelnen Datei gespeichert.
- Das bringt uns an eine gewisse Grenze, für die Komplexität von den Applikationen, die wir bauen können.
- Nach ein paar Tausend Zeile Kode in einer einzelnen Datei und vielleicht ein paar Duzend Funktionen wird es sehr schwer, irgendeine Übersicht zu behalten.

Einleitung

- Es ist Ihnen vielleicht nicht aufgefallen, aber wir haben gerade wieder einen großen Schritt in unseren Programmierfähigkeiten gemacht.
- Wir sind von einzelnen Programmen, die nur aus einem Block von Kode bestehen, zu modularen Programmen fortgeschritten.
- Wir können nun Kode wiederverwenden.
- Als wir unsere Reise begonnen haben, haben wir unsere Befehle einen nach dem anderen in den Python-Interpreter eingetippt und ausgeführt.
- Dann haben wir den Kode in Dateien geschrieben, wodurch wir dann unsere Programme mehrmals ausführen konnten.
- Jetzt können wir unseren Kode in Funktionen strukturieren, die wir mit Docstrings und Type Hints erklären können.
- Jetzt können wir schon nützliche und vernünftig große Programme schreiben.
- Aber bis jetzt sind unsere gesamten Programme noch in einzelnen Datei gespeichert.
- Das bringt uns an eine gewisse Grenze, für die Komplexität von den Applikationen, die wir bauen können.
- Nach ein paar Tausend Zeile Kode in einer einzelnen Datei und vielleicht ein paar Duzend Funktionen wird es sehr schwer, irgendeine Übersicht zu behalten.

Module und Pakete

- Wir lernen jetzt also, wie wir Kode auf mehrere Dateien verteilen können.

Module und Pakete

- Wir lernen jetzt also, wie wir Kode auf mehrere Dateien verteilen können.
- Das beantwortet zwei wichtige Fragen.

Module und Pakete

- Wir lernen jetzt also, wie wir Kode auf mehrere Dateien verteilen können.
- Das beantwortet zwei wichtige Fragen
  1. Wie können wir es vermeiden, unsere Applikationen als einzelne große, unstrukturierte Dateien zu schreiben, die wir unmöglich auf lange Zeit warten können?

Module und Pakete

- Wir lernen jetzt also, wie wir Kode auf mehrere Dateien verteilen können.
- Das beantwortet zwei wichtige Fragen
  1. Wie können wir es vermeiden, unsere Applikationen als einzelne große, unstrukturierte Dateien zu schreiben, die wir unmöglich auf lange Zeit warten können?
  2. Wie können wir unsere Applikationen in kleinere Einheiten zerlegen, die wir einzeln Testen, Verbessern, und Warten können und vielleicht sogar in verschiedenen Kontexten verwenden können?

Module und Pakete

- Wir lernen jetzt also, wie wir Kode auf mehrere Dateien verteilen können.
- Das beantwortet zwei wichtige Fragen
  1. Wie können wir es vermeiden, unsere Applikationen als einzelne große, unstrukturierte Dateien zu schreiben, die wir unmöglich auf lange Zeit warten können?
  2. Wie können wir unsere Applikationen in kleinere Einheiten zerlegen, die wir einzeln Testen, Verbessern, und Warten können und vielleicht sogar in verschiedenen Kontexten verwenden können?
- Ein großer Teil der Antwort sind *Module* und *Pakete*[25].

Module und Pakete

- Wir lernen jetzt also, wie wir Kode auf mehrere Dateien verteilen können.
- Das beantwortet zwei wichtige Fragen
  1. Wie können wir es vermeiden, unsere Applikationen als einzelne große, unstrukturierte Dateien zu schreiben, die wir unmöglich auf lange Zeit warten können?
  2. Wie können wir unsere Applikationen in kleinere Einheiten zerlegen, die wir einzeln Testen, Verbessern, und Warten können und vielleicht sogar in verschiedenen Kontexten verwenden können?
- Ein großer Teil der Antwort sind *Module* und *Pakete*[25].
- Hier, ist ein *Modul* im Grunde eine Python-Datei und ein *Paket* ist ein Verzeichnis, das solche Dateien beinhaltet.

Module und Pakete

- Wir lernen jetzt also, wie wir Kode auf mehrere Dateien verteilen können.
- Das beantwortet zwei wichtige Fragen
  1. Wie können wir es vermeiden, unsere Applikationen als einzelne große, unstrukturierte Dateien zu schreiben, die wir unmöglich auf lange Zeit warten können?
  2. Wie können wir unsere Applikationen in kleinere Einheiten zerlegen, die wir einzeln Testen, Verbessern, und Warten können und vielleicht sogar in verschiedenen Kontexten verwenden können?
- Ein großer Teil der Antwort sind *Module* und *Pakete*[25].
- Hier, ist ein *Modul* im Grunde eine Python-Datei und ein *Paket* ist ein Verzeichnis, das solche Dateien beinhaltet.
- Wie in [25] steht, müssen Module nicht unbedingt Dateien seien und Pakete können auch anders erstellt werden ... wir bleiben hier aber bei den einfachen Definitionen.

Module und Pakete

- Wir lernen jetzt also, wie wir Kode auf mehrere Dateien verteilen können.
- Das beantwortet zwei wichtige Fragen
  1. Wie können wir es vermeiden, unsere Applikationen als einzelne große, unstrukturierte Dateien zu schreiben, die wir unmöglich auf lange Zeit warten können?
  2. Wie können wir unsere Applikationen in kleinere Einheiten zerlegen, die wir einzeln Testen, Verbessern, und Warten können und vielleicht sogar in verschiedenen Kontexten verwenden können?
- Ein großer Teil der Antwort sind *Module* und *Pakete*[25].
- Hier, ist ein *Modul* im Grunde eine Python-Datei und ein *Paket* ist ein Verzeichnis, das solche Dateien beinhaltet.
- Wie in [25] steht, müssen Module nicht unbedingt Dateien seien und Pakete können auch anders erstellt werden ... wir bleiben hier aber bei den einfachen Definitionen.
- Wir haben ja auch schon mit Modulen gearbeitet, z. B. dem Modul `math`.

Beispiel

Unser Eigenes Modul

- Wir haben ja auch schon mit Modulen gearbeitet, z. B. dem Modul `math`.

Unser Eigenes Modul

- Wir haben ja auch schon mit Modulen gearbeitet, z. B. dem Modul `math`.
- Dieses Modul ist im Grunde eine Kollektion mathematischer Funktionen.

Unser Eigenes Modul

- Wir haben ja auch schon mit Modulen gearbeitet, z. B. dem Modul `math`.
- Dieses Modul ist im Grunde eine Kollektion mathematischer Funktionen.
- Wir haben ja auch selber schon ein paar mathematische Funktionen implementiert.

Unser Eigenes Modul

- Wir haben ja auch schon mit Modulen gearbeitet, z. B. dem Modul `math`.
- Dieses Modul ist im Grunde eine Kollektion mathematischer Funktionen.
- Wir haben ja auch selber schon ein paar mathematische Funktionen implementiert.
- Tuen wir also ein paar davon in ein Modul!

Unser Eigenes Modul

- Wir haben ja auch schon mit Modulen gearbeitet, z. B. dem Modul `math`.
- Dieses Modul ist im Grunde eine Kollektion mathematischer Funktionen.
- Wir haben ja auch selber schon ein paar mathematische Funktionen implementiert.
- Tuen wir also ein paar davon in ein Modul!
- Wir erstellen die Datei `my_math.py` und tuen zwei Funktionen hinein.

```python
"""A module with mathematics routines."""

from math import isclose  # Checks if two float numbers are similar.


def factorial(a: int) -> int:  # 1 `int` parameter and `int` result
    """
    Compute the factorial of a positive integer `a`.

    :param a: the number to compute the factorial of
    :return: the factorial of `a`, i.e., `a!`.
    """
    product: int = 1  # Initialize `product` as `1`.
    for i in range(2, a + 1):  # `i` goes from `2` to `a`.
        product *= i  # Multiply `i` to the product.
    return product  # Return the product, which now is the factorial.


def sqrt(number: float) -> float:
    """
    Compute the square root of a given `number`.

    :param number: The number to compute the square root of.
    :return: A value `v` such that `v * v` is approximately `number`.
    """
    guess: float = 1.0       # This will hold the current guess.
    old_guess: float = 0.0  # 0.0 is just a dummy value != guess.
    while not isclose(old_guess, guess):   # Repeat until no change.
        old_guess = guess   # The current guess becomes the old guess.
        guess = 0.5 * (guess + number / guess)  # The new guess.
    return guess
```

Unser Eigenes Modul

- Dieses Modul ist im Grunde eine Kollektion mathematischer Funktionen.
- Wir haben ja auch selber schon ein paar mathematische Funktionen implementiert.
- Tuen wir also ein paar davon in ein Modul!
- Wir erstellen die Datei `my_math.py` und tuen zwei Funktionen hinein.

Gute Praxis

Die Namen von Paketen und Modulen sollen kurz und in Kleinbuchstaben geschrieben werden. Unterstriche können verwendet werden, um die Lesbarkeit zu verbessern.[31]

Unser Eigenes Modul

- Wir haben ja auch selber schon ein paar mathematische Funktionen implementiert.
- Tuen wir also ein paar davon in ein Modul!
- Wir erstellen die Datei `my_math.py` und tuen zwei Funktionen hinein:
- Die Function `factorial` und eine neue Funktion namens `sqrt`.

```python
"""A module with mathematics routines."""

from math import isclose  # Checks if two float numbers are similar.


def factorial(a: int) -> int:  # 1 `int` parameter and `int` result
    """
    Compute the factorial of a positive integer `a`.

    :param a: the number to compute the factorial of
    :return: the factorial of `a`, i.e., `a!`.
    """
    product: int = 1  # Initialize `product` as `1`.
    for i in range(2, a + 1):  # `i` goes from `2` to `a`.
        product *= i  # Multiply `i` to the product.
    return product  # Return the product, which now is the factorial.


def sqrt(number: float) -> float:
    """
    Compute the square root of a given `number`.

    :param number: The number to compute the square root of.
    :return: A value `v` such that `v * v` is approximately `number`.
    """
    guess: float = 1.0       # This will hold the current guess.
    old_guess: float = 0.0  # 0.0 is just a dummy value != guess.
    while not isclose(old_guess, guess):   # Repeat until no change.
        old_guess = guess   # The current guess becomes the old guess.
        guess = 0.5 * (guess + number / guess)  # The new guess.
    return guess
```

Unser Eigenes Modul

- Tuen wir also ein paar davon in ein Modul!
- Wir erstellen die Datei `my_math.py` und tuen zwei Funktionen hinein:
- Die Function `factorial` und eine neue Funktion namens `sqrt`.
- Die `sqrt`-Function basically ist im Grunde unser Kode von Einheit 25 mit unserer Implementierung von Heron's Methode zur Berechnung der Quadratwurzel.

```python
"""A module with mathematics routines."""

from math import isclose  # Checks if two float numbers are similar.


def factorial(a: int) -> int:  # 1 `int` parameter and `int` result
    """
    Compute the factorial of a positive integer `a`.

    :param a: the number to compute the factorial of
    :return: the factorial of `a`, i.e., `a!`.
    """
    product: int = 1  # Initialize `product` as `1`.
    for i in range(2, a + 1):  # `i` goes from `2` to `a`.
        product *= i  # Multiply `i` to the product.
    return product  # Return the product, which now is the factorial.


def sqrt(number: float) -> float:
    """
    Compute the square root of a given `number`.

    :param number: The number to compute the square root of.
    :return: A value `v` such that `v * v` is approximately `number`.
    """
    guess: float = 1.0       # This will hold the current guess.
    old_guess: float = 0.0  # 0.0 is just a dummy value != guess.
    while not isclose(old_guess, guess):   # Repeat until no change.
        old_guess = guess   # The current guess becomes the old guess.
        guess = 0.5 * (guess + number / guess)  # The new guess.
    return guess
```

Unser Eigenes Modul

- Wir erstellen die Datei `my_math.py` und tuen zwei Funktionen hinein:
- Die Function `factorial` und eine neue Funktion namens `sqrt`.
- Die `sqrt`-Function basically ist im Grunde unser Kode von Einheit 25 mit unserer Implementierung von Heron's Methode zur Berechnung der Quadratwurzel.
- Nun kommt jedoch `number` als Parameter in unsere Funktion hinein.

```python
"""A module with mathematics routines."""

from math import isclose  # Checks if two float numbers are similar.


def factorial(a: int) -> int:  # 1 `int` parameter and `int` result
    """
    Compute the factorial of a positive integer `a`.

    :param a: the number to compute the factorial of
    :return: the factorial of `a`, i.e., `a!`.
    """
    product: int = 1  # Initialize `product` as `1`.
    for i in range(2, a + 1):  # `i` goes from `2` to `a`.
        product *= i  # Multiply `i` to the product.
    return product  # Return the product, which now is the factorial.


def sqrt(number: float) -> float:
    """
    Compute the square root of a given `number`.

    :param number: The number to compute the square root of.
    :return: A value `v` such that `v * v` is approximately `number`.
    """
    guess: float = 1.0       # This will hold the current guess.
    old_guess: float = 0.0  # 0.0 is just a dummy value != guess.
    while not isclose(old_guess, guess):   # Repeat until no change.
        old_guess = guess   # The current guess becomes the old guess.
        guess = 0.5 * (guess + number / guess)  # The new guess.
    return guess
```

Unser Eigenes Modul

- Die Function `factorial` und eine neue Funktion namens `sqrt`.
- Die `sqrt`-Function basically ist im Grunde unser Kode von Einheit 25 mit unserer Implementierung von Heron's Methode zur Berechnung der Quadratwurzel.
- Nun kommt jedoch `number` als Parameter in unsere Funktion hinein.
- Unser neues Modul has the name `my_math`, weil es in der Datei `my_math.py` gespeichert ist.

```python
"""A module with mathematics routines."""

from math import isclose  # Checks if two float numbers are similar.


def factorial(a: int) -> int:  # 1 `int` parameter and `int` result
    """
    Compute the factorial of a positive integer `a`.

    :param a: the number to compute the factorial of
    :return: the factorial of `a`, i.e., `a!`.
    """
    product: int = 1  # Initialize `product` as `1`.
    for i in range(2, a + 1):  # `i` goes from `2` to `a`.
        product *= i  # Multiply `i` to the product.
    return product  # Return the product, which now is the factorial.


def sqrt(number: float) -> float:
    """
    Compute the square root of a given `number`.

    :param number: The number to compute the square root of.
    :return: A value `v` such that `v * v` is approximately `number`.
    """
    guess: float = 1.0       # This will hold the current guess.
    old_guess: float = 0.0  # 0.0 is just a dummy value != guess.
    while not isclose(old_guess, guess):   # Repeat until no change.
        old_guess = guess   # The current guess becomes the old guess.
        guess = 0.5 * (guess + number / guess)  # The new guess.
    return guess
```

Unser Eigenes Modul

- Die Function `factorial` und eine neue Funktion namens `sqrt`.
- Die `sqrt`-Function basically ist im Grunde unser Kode von Einheit 25 mit unserer Implementierung von Heron's Methode zur Berechnung der Quadratwurzel.
- Nun kommt jedoch `number` als Parameter in unsere Funktion hinein.
- Unser neues Modul has the name `my_math`, weil es in der Datei `my_math.py` gespeichert ist.
- Es sieht im Grunde nicht anders aus, als was wir bisher gemacht haben.

```python
"""A module with mathematics routines."""

from math import isclose  # Checks if two float numbers are similar.


def factorial(a: int) -> int:  # 1 `int` parameter and `int` result
    """
    Compute the factorial of a positive integer `a`.

    :param a: the number to compute the factorial of
    :return: the factorial of `a`, i.e., `a!`.
    """
    product: int = 1  # Initialize `product` as `1`.
    for i in range(2, a + 1):  # `i` goes from `2` to `a`.
        product *= i  # Multiply `i` to the product.
    return product  # Return the product, which now is the factorial.


def sqrt(number: float) -> float:
    """
    Compute the square root of a given `number`.

    :param number: The number to compute the square root of.
    :return: A value `v` such that `v * v` is approximately `number`.
    """
    guess: float = 1.0       # This will hold the current guess.
    old_guess: float = 0.0  # 0.0 is just a dummy value != guess.
    while not isclose(old_guess, guess):   # Repeat until no change.
        old_guess = guess   # The current guess becomes the old guess.
        guess = 0.5 * (guess + number / guess)  # The new guess.
    return guess
```

Unser Eigenes Modul

- Die `sqrt`-Function basically ist im Grunde unser Kode von Einheit 25 mit unserer Implementierung von Heron's Methode zur Berechnung der Quadratwurzel.
- Nun kommt jedoch `number` als Parameter in unsere Funktion hinein.
- Unser neues Modul has the name `my_math`, weil es in der Datei `my_math.py` gespeichert ist.
- Es sieht im Grunde nicht anders aus, als was wir bisher gemacht haben.
- Ein Unterschied ist, dass es *nichts* macht.

```python
"""A module with mathematics routines."""

from math import isclose  # Checks if two float numbers are similar.


def factorial(a: int) -> int:  # 1 `int` parameter and `int` result
    """
    Compute the factorial of a positive integer `a`.

    :param a: the number to compute the factorial of
    :return: the factorial of `a`, i.e., `a!`.
    """
    product: int = 1  # Initialize `product` as `1`.
    for i in range(2, a + 1):  # `i` goes from `2` to `a`.
        product *= i  # Multiply `i` to the product.
    return product  # Return the product, which now is the factorial.


def sqrt(number: float) -> float:
    """
    Compute the square root of a given `number`.

    :param number: The number to compute the square root of.
    :return: A value `v` such that `v * v` is approximately `number`.
    """
    guess: float = 1.0       # This will hold the current guess.
    old_guess: float = 0.0  # 0.0 is just a dummy value != guess.
    while not isclose(old_guess, guess):   # Repeat until no change.
        old_guess = guess   # The current guess becomes the old guess.
        guess = 0.5 * (guess + number / guess)  # The new guess.
    return guess
```

Unser Eigenes Modul

- Nun kommt jedoch `number` als Parameter in unsere Funktion hinein.
- Unser neues Modul has the name `my_math`, weil es in der Datei `my_math.py` gespeichert ist.
- Es sieht im Grunde nicht anders aus, als was wir bisher gemacht haben.
- Ein Unterschied ist, dass es *nichts* macht.
- In der Datei erstellen wir zwei Funktionen.

```python
"""A module with mathematics routines."""

from math import isclose  # Checks if two float numbers are similar.


def factorial(a: int) -> int:  # 1 `int` parameter and `int` result
    """
    Compute the factorial of a positive integer `a`.

    :param a: the number to compute the factorial of
    :return: the factorial of `a`, i.e., `a!`.
    """
    product: int = 1  # Initialize `product` as `1`.
    for i in range(2, a + 1):  # `i` goes from `2` to `a`.
        product *= i  # Multiply `i` to the product.
    return product  # Return the product, which now is the factorial.


def sqrt(number: float) -> float:
    """
    Compute the square root of a given `number`.

    :param number: The number to compute the square root of.
    :return: A value `v` such that `v * v` is approximately `number`.
    """
    guess: float = 1.0       # This will hold the current guess.
    old_guess: float = 0.0  # 0.0 is just a dummy value != guess.
    while not isclose(old_guess, guess):   # Repeat until no change.
        old_guess = guess   # The current guess becomes the old guess.
        guess = 0.5 * (guess + number / guess)  # The new guess.
    return guess
```

Unser Eigenes Modul

- Unser neues Modul has the name `my_math`, weil es in der Datei `my_math.py` gespeichert ist.
- Es sieht im Grunde nicht anders aus, als was wir bisher gemacht haben.
- Ein Unterschied ist, dass es *nichts* macht.
- In der Datei erstellen wir zwei Funktionen.
- Wir rufen sie aber nicht auf und führen auch sonst keinen Kode aktiv aus.

```python
"""A module with mathematics routines."""

from math import isclose  # Checks if two float numbers are similar.


def factorial(a: int) -> int:  # 1 `int` parameter and `int` result
    """
    Compute the factorial of a positive integer `a`.

    :param a: the number to compute the factorial of
    :return: the factorial of `a`, i.e., `a!`.
    """
    product: int = 1  # Initialize `product` as `1`.
    for i in range(2, a + 1):  # `i` goes from `2` to `a`.
        product *= i  # Multiply `i` to the product.
    return product  # Return the product, which now is the factorial.


def sqrt(number: float) -> float:
    """
    Compute the square root of a given `number`.

    :param number: The number to compute the square root of.
    :return: A value `v` such that `v * v` is approximately `number`.
    """
    guess: float = 1.0       # This will hold the current guess.
    old_guess: float = 0.0  # 0.0 is just a dummy value != guess.
    while not isclose(old_guess, guess):   # Repeat until no change.
        old_guess = guess   # The current guess becomes the old guess.
        guess = 0.5 * (guess + number / guess)  # The new guess.
    return guess
```

Unser Eigenes Modul

- Es sieht im Grunde nicht anders aus, als was wir bisher gemacht haben.
- Ein Unterschied ist, dass es *nichts* macht.
- In der Datei erstellen wir zwei Funktionen.
- Wir rufen sie aber nicht auf und führen auch sonst keinen Kode aktiv aus.
- Das ist der Sinn des Moduls.

```python
"""A module with mathematics routines."""

from math import isclose  # Checks if two float numbers are similar.


def factorial(a: int) -> int:  # 1 `int` parameter and `int` result
    """
    Compute the factorial of a positive integer `a`.

    :param a: the number to compute the factorial of
    :return: the factorial of `a`, i.e., `a!`.
    """
    product: int = 1  # Initialize `product` as `1`.
    for i in range(2, a + 1):  # `i` goes from `2` to `a`.
        product *= i  # Multiply `i` to the product.
    return product  # Return the product, which now is the factorial.


def sqrt(number: float) -> float:
    """
    Compute the square root of a given `number`.

    :param number: The number to compute the square root of.
    :return: A value `v` such that `v * v` is approximately `number`.
    """
    guess: float = 1.0       # This will hold the current guess.
    old_guess: float = 0.0  # 0.0 is just a dummy value != guess.
    while not isclose(old_guess, guess):   # Repeat until no change.
        old_guess = guess   # The current guess becomes the old guess.
        guess = 0.5 * (guess + number / guess)  # The new guess.
    return guess
```

Unser Eigenes Modul

- Ein Unterschied ist, dass es *nichts* macht.
- In der Datei erstellen wir zwei Funktionen.
- Wir rufen sie aber nicht auf und führen auch sonst keinen Kode aktiv aus.
- Das ist der Sinn des Moduls.
- Es stellt zwei Funktionen zur Verfügung.

```python
"""A module with mathematics routines."""

from math import isclose  # Checks if two float numbers are similar.


def factorial(a: int) -> int:  # 1 `int` parameter and `int` result
    """
    Compute the factorial of a positive integer `a`.

    :param a: the number to compute the factorial of
    :return: the factorial of `a`, i.e., `a!`.
    """
    product: int = 1  # Initialize `product` as `1`.
    for i in range(2, a + 1):  # `i` goes from `2` to `a`.
        product *= i  # Multiply `i` to the product.
    return product  # Return the product, which now is the factorial.


def sqrt(number: float) -> float:
    """
    Compute the square root of a given `number`.

    :param number: The number to compute the square root of.
    :return: A value `v` such that `v * v` is approximately `number`.
    """
    guess: float = 1.0       # This will hold the current guess.
    old_guess: float = 0.0  # 0.0 is just a dummy value != guess.
    while not isclose(old_guess, guess):   # Repeat until no change.
        old_guess = guess   # The current guess becomes the old guess.
        guess = 0.5 * (guess + number / guess)  # The new guess.
    return guess
```

Unser Eigenes Modul

- In der Datei erstellen wir zwei Funktionen.
- Wir rufen sie aber nicht auf und führen auch sonst keinen Kode aktiv aus.
- Das ist der Sinn des Moduls.
- Es stellt zwei Funktionen zur Verfügung.
- Diese können wir nun an anderer Stelle verwenden.

```python
"""A module with mathematics routines."""

from math import isclose  # Checks if two float numbers are similar.


def factorial(a: int) -> int:  # 1 `int` parameter and `int` result
    """
    Compute the factorial of a positive integer `a`.

    :param a: the number to compute the factorial of
    :return: the factorial of `a`, i.e., `a!`.
    """
    product: int = 1  # Initialize `product` as `1`.
    for i in range(2, a + 1):  # `i` goes from `2` to `a`.
        product *= i  # Multiply `i` to the product.
    return product  # Return the product, which now is the factorial.


def sqrt(number: float) -> float:
    """
    Compute the square root of a given `number`.

    :param number: The number to compute the square root of.
    :return: A value `v` such that `v * v` is approximately `number`.
    """
    guess: float = 1.0       # This will hold the current guess.
    old_guess: float = 0.0  # 0.0 is just a dummy value != guess.
    while not isclose(old_guess, guess):   # Repeat until no change.
        old_guess = guess   # The current guess becomes the old guess.
        guess = 0.5 * (guess + number / guess)  # The new guess.
    return guess
```

Unser Eigenes Modul

- Wir rufen sie aber nicht auf und führen auch sonst keinen Kode aktiv aus.
- Das ist der Sinn des Moduls.
- Es stellt zwei Funktionen zur Verfügung.
- Diese können wir nun an anderer Stelle verwenden.
- Und Datei `use_my_math.py` ist wo wir sie verwenden.

```python
"""Using the mathematics module."""

from my_math import factorial, sqrt  # Import our two functions.

print(f"6!={factorial(6)}")  # Use the `factorial` function.
print(f"\u221A3={sqrt(3.0)}")   # Use the `sqrt` function.
print(f"\u221A(6!*1.5)={sqrt(factorial(6) * 1.5)}")  # Use both.

print("We now use Liu Hui's Method to Approximate \u03c0.")
e: int = 6  # the number of edges: We start with a hexagon, i.e., e=6.
s: float = 1.0  # the side length: Initially 1, i.e., radius is also 1.
for _ in range(6):
    print(f"{e} edges, side length={s} give us \u03c0\u2248{e * s / 2}.")
    e *= 2  # We double the number of edges...
    s = sqrt(2 - sqrt(4 - (s ** 2)))  # ...and recompute the side length.
```

↓ `python3 use_my_math.py` ↓

```
6!=720
√3=1.7320508075688772
√(6!*1.5)=32.863353450309965
We now use Liu Hui's Method to Approximate π.
6 edges, side length=1.0 give us π≈3.0.
12 edges, side length=0.5176380902050417 give us π≈3.10582854123025.
24 edges, side length=0.2610523844401035 give us π≈3.1326286132812418.
48 edges, side length=0.13080625846028637 give us π≈3.139350203046873.
96 edges, side length=0.0654381656435527 give us π≈3.14103195089053.
192 edges, side length=0.03272346325297234 give us π≈3.1414524722853443.
```

Unser Eigenes Modul

- Das ist der Sinn des Moduls.
- Es stellt zwei Funktionen zur Verfügung.
- Diese können wir nun an anderer Stelle verwenden.
- Und Datei `use_my_math.py` ist wo wir sie verwenden.
- In dieser Datei wollen wir unsere beiden Funktionen `factorial` und `sqrt` vom Modul `my_math` benutzen.

```python
"""Using the mathematics module."""

from my_math import factorial, sqrt  # Import our two functions.

print(f"6!={factorial(6)}")  # Use the `factorial` function.
print(f"\u221A3={sqrt(3.0)}")   # Use the `sqrt` function.
print(f"\u221A(6!*1.5)={sqrt(factorial(6) * 1.5)}")  # Use both.

print("We now use Liu Hui's Method to Approximate \u03c0.")
e: int = 6  # the number of edges: We start with a hexagon, i.e., e=6.
s: float = 1.0  # the side length: Initially 1, i.e., radius is also 1.
for _ in range(6):
    print(f"{e} edges, side length={s} give us \u03c0\u2248{e * s / 2}.")
    e *= 2  # We double the number of edges...
    s = sqrt(2 - sqrt(4 - (s ** 2)))  # ...and recompute the side length.
```

↓ `python3 use_my_math.py` ↓

```
6!=720
√3=1.7320508075688772
√(6!*1.5)=32.863353450309965
We now use Liu Hui's Method to Approximate π.
6 edges, side length=1.0 give us π≈3.0.
12 edges, side length=0.5176380902050417 give us π≈3.10582854123025.
24 edges, side length=0.2610523844401035 give us π≈3.1326286132812418.
48 edges, side length=0.13080625846028637 give us π≈3.139350203046873.
96 edges, side length=0.0654381656435527 give us π≈3.14103195089053.
192 edges, side length=0.03272346325297234 give us π≈3.1414524722853443.
```

Unser Eigenes Modul

- Es stellt zwei Funktionen zur Verfügung.
- Diese können wir nun an anderer Stelle verwenden.
- Und Datei `use_my_math.py` ist wo wir sie verwenden.
- In dieser Datei wollen wir unsere beiden Funktionen `factorial` und `sqrt` vom Modul `my_math` benutzen.
- Wir müssen dem Python-Interpreter also sagen, wo es diese finden kann.

```python
"""Using the mathematics module."""

from my_math import factorial, sqrt  # Import our two functions.

print(f"6!={factorial(6)}")  # Use the `factorial` function.
print(f"\u221A3={sqrt(3.0)}")   # Use the `sqrt` function.
print(f"\u221A(6!*1.5)={sqrt(factorial(6) * 1.5)}")  # Use both.

print("We now use Liu Hui's Method to Approximate \u03c0.")
e: int = 6  # the number of edges: We start with a hexagon, i.e., e=6.
s: float = 1.0  # the side length: Initially 1, i.e., radius is also 1.
for _ in range(6):
    print(f"{e} edges, side length={s} give us \u03c0\u2248{e * s / 2}.")
    e *= 2  # We double the number of edges...
    s = sqrt(2 - sqrt(4 - (s ** 2)))  # ...and recompute the side length.
```

↓ `python3 use_my_math.py` ↓

```
6!=720
√3=1.7320508075688772
√(6!*1.5)=32.863353450309965
We now use Liu Hui's Method to Approximate π.
6 edges, side length=1.0 give us π≈3.0.
12 edges, side length=0.5176380902050417 give us π≈3.10582854123025.
24 edges, side length=0.2610523844401035 give us π≈3.1326286132812418.
48 edges, side length=0.13080625846028637 give us π≈3.139350203046873.
96 edges, side length=0.0654381656435527 give us π≈3.14103195089053.
192 edges, side length=0.03272346325297234 give us π≈3.1414524722853443.
```

Unser Eigenes Modul

- Diese können wir nun an anderer Stelle verwenden.
- Und Datei `use_my_math.py` ist wo wir sie verwenden.
- In dieser Datei wollen wir unsere beiden Funktionen `factorial` und `sqrt` vom Modul `my_math` benutzen.
- Wir müssen dem Python-Interpreter also sagen, wo es diese finden kann.
- Wir tuen das, in dem wir schreiben `from my_math import factorial, sqrt`.

```python
"""Using the mathematics module."""

from my_math import factorial, sqrt  # Import our two functions.

print(f"6!={factorial(6)}")  # Use the `factorial` function.
print(f"\u221A3={sqrt(3.0)}")   # Use the `sqrt` function.
print(f"\u221A(6!*1.5)={sqrt(factorial(6) * 1.5)}")  # Use both.

print("We now use Liu Hui's Method to Approximate \u03c0.")
e: int = 6  # the number of edges: We start with a hexagon, i.e., e=6.
s: float = 1.0  # the side length: Initially 1, i.e., radius is also 1.
for _ in range(6):
    print(f"{e} edges, side length={s} give us \u03c0\u2248{e * s / 2}.")
    e *= 2  # We double the number of edges...
    s = sqrt(2 - sqrt(4 - (s ** 2)))  # ...and recompute the side length.
```

↓ `python3 use_my_math.py` ↓

```
6!=720
√3=1.7320508075688772
√(6!*1.5)=32.863353450309965
We now use Liu Hui's Method to Approximate π.
6 edges, side length=1.0 give us π≈3.0.
12 edges, side length=0.5176380902050417 give us π≈3.10582854123025.
24 edges, side length=0.2610523844401035 give us π≈3.1326286132812418.
48 edges, side length=0.13080625846028637 give us π≈3.139350203046873.
96 edges, side length=0.0654381656435527 give us π≈3.14103195089053.
192 edges, side length=0.03272346325297234 give us π≈3.1414524722853443.
```

Unser Eigenes Modul

- Und Datei `use_my_math.py` ist wo wir sie verwenden.
- In dieser Datei wollen wir unsere beiden Funktionen `factorial` und `sqrt` vom Modul `my_math` benutzen.
- Wir müssen dem Python-Interpreter also sagen, wo es diese finden kann.
- Wir tuen das, in dem wir schreiben `from my_math import factorial, sqrt`.
- Die Bedeutung ist ziemlich klar.

```python
"""Using the mathematics module."""

from my_math import factorial, sqrt  # Import our two functions.

print(f"6!={factorial(6)}")  # Use the `factorial` function.
print(f"\u221A3={sqrt(3.0)}")   # Use the `sqrt` function.
print(f"\u221A(6!*1.5)={sqrt(factorial(6) * 1.5)}")  # Use both.

print("We now use Liu Hui's Method to Approximate \u03c0.")
e: int = 6  # the number of edges: We start with a hexagon, i.e., e=6.
s: float = 1.0  # the side length: Initially 1, i.e., radius is also 1.
for _ in range(6):
    print(f"{e} edges, side length={s} give us \u03c0\u2248{e * s / 2}.")
    e *= 2  # We double the number of edges...
    s = sqrt(2 - sqrt(4 - (s ** 2)))  # ...and recompute the side length.
```

↓ `python3 use_my_math.py` ↓

```
6!=720
√3=1.7320508075688772
√(6!*1.5)=32.863353450309965
We now use Liu Hui's Method to Approximate π.
6 edges, side length=1.0 give us π≈3.0.
12 edges, side length=0.5176380902050417 give us π≈3.10582854123025.
24 edges, side length=0.2610523844401035 give us π≈3.1326286132812418.
48 edges, side length=0.13080625846028637 give us π≈3.139350203046873.
96 edges, side length=0.0654381656435527 give us π≈3.14103195089053.
192 edges, side length=0.03272346325297234 give us π≈3.1414524722853443.
```

Unser Eigenes Modul

- In dieser Datei wollen wir unsere beiden Funktionen `factorial` und `sqrt` vom Modul `my_math` benutzen.
- Wir müssen dem Python-Interpreter also sagen, wo es diese finden kann.
- Wir tuen das, in dem wir schreiben `from my_math import factorial, sqrt`.
- Die Bedeutung ist ziemlich klar.
- Es gibt ein Modul `my_math` von dem wir die Funktionen `factorial` holen wollen `sqrt`.

```python
"""Using the mathematics module."""

from my_math import factorial, sqrt  # Import our two functions.

print(f"6!={factorial(6)}")  # Use the `factorial` function.
print(f"\u221A3={sqrt(3.0)}")   # Use the `sqrt` function.
print(f"\u221A(6!*1.5)={sqrt(factorial(6) * 1.5)}")  # Use both.

print("We now use Liu Hui's Method to Approximate \u03c0.")
e: int = 6  # the number of edges: We start with a hexagon, i.e., e=6.
s: float = 1.0  # the side length: Initially 1, i.e., radius is also 1.
for _ in range(6):
    print(f"{e} edges, side length={s} give us \u03c0\u2248{e * s / 2}.")
    e *= 2  # We double the number of edges...
    s = sqrt(2 - sqrt(4 - (s ** 2)))  # ...and recompute the side length.
```

↓ `python3 use_my_math.py` ↓

```
6!=720
√3=1.7320508075688772
√(6!*1.5)=32.863353450309965
We now use Liu Hui's Method to Approximate π.
6 edges, side length=1.0 give us π≈3.0.
12 edges, side length=0.5176380902050417 give us π≈3.10582854123025.
24 edges, side length=0.2610523844401035 give us π≈3.1326286132812418.
48 edges, side length=0.13080625846028637 give us π≈3.139350203046873.
96 edges, side length=0.0654381656435527 give us π≈3.14103195089053.
192 edges, side length=0.03272346325297234 give us π≈3.1414524722853443.
```

Unser Eigenes Modul

- Wir müssen dem Python-Interpreter also sagen, wo es diese finden kann.
- Wir tuen das, in dem wir schreiben `from my_math import factorial, sqrt`.
- Die Bedeutung ist ziemlich klar.
- Es gibt ein Modul `my_math` von dem wir die Funktionen `factorial` holen wollen `sqrt`.
- Der Python-Interpreter kennt viele Module.

```python
"""Using the mathematics module."""

from my_math import factorial, sqrt  # Import our two functions.

print(f"6!={factorial(6)}")  # Use the `factorial` function.
print(f"\u221A3={sqrt(3.0)}")   # Use the `sqrt` function.
print(f"\u221A(6!*1.5)={sqrt(factorial(6) * 1.5)}")  # Use both.

print("We now use Liu Hui's Method to Approximate \u03c0.")
e: int = 6  # the number of edges: We start with a hexagon, i.e., e=6.
s: float = 1.0  # the side length: Initially 1, i.e., radius is also 1.
for _ in range(6):
    print(f"{e} edges, side length={s} give us \u03c0\u2248{e * s / 2}.")
    e *= 2  # We double the number of edges...
    s = sqrt(2 - sqrt(4 - (s ** 2)))  # ...and recompute the side length.
```

↓ `python3 use_my_math.py` ↓

```
6!=720
√3=1.7320508075688772
√(6!*1.5)=32.863353450309965
We now use Liu Hui's Method to Approximate π.
6 edges, side length=1.0 give us π≈3.0.
12 edges, side length=0.5176380902050417 give us π≈3.10582854123025.
24 edges, side length=0.2610523844401035 give us π≈3.1326286132812418.
48 edges, side length=0.13080625846028637 give us π≈3.139350203046873.
96 edges, side length=0.0654381656435527 give us π≈3.14103195089053.
192 edges, side length=0.03272346325297234 give us π≈3.1414524722853443.
```

Unser Eigenes Modul

- Wir tuen das, in dem wir schreiben `from my_math import factorial, sqrt`.
- Die Bedeutung ist ziemlich klar.
- Es gibt ein Modul `my_math` von dem wir die Funktionen `factorial` holen wollen `sqrt`.
- Der Python-Interpreter kennt viele Module.
- Mehrere Module kommen schon mit der Python-Installation mit, wie z. B. `math`.

```python
"""Using the mathematics module."""

from my_math import factorial, sqrt  # Import our two functions.

print(f"6!={factorial(6)}")  # Use the `factorial` function.
print(f"\u221A3={sqrt(3.0)}")   # Use the `sqrt` function.
print(f"\u221A(6!*1.5)={sqrt(factorial(6) * 1.5)}")  # Use both.

print("We now use Liu Hui's Method to Approximate \u03c0.")
e: int = 6  # the number of edges: We start with a hexagon, i.e., e=6.
s: float = 1.0  # the side length: Initially 1, i.e., radius is also 1.
for _ in range(6):
    print(f"{e} edges, side length={s} give us \u03c0\u2248{e * s / 2}.")
    e *= 2  # We double the number of edges...
    s = sqrt(2 - sqrt(4 - (s ** 2)))  # ...and recompute the side length.
```

↓ `python3 use_my_math.py` ↓

```
6!=720
√3=1.7320508075688772
√(6!*1.5)=32.863353450309965
We now use Liu Hui's Method to Approximate π.
6 edges, side length=1.0 give us π≈3.0.
12 edges, side length=0.5176380902050417 give us π≈3.10582854123025.
24 edges, side length=0.2610523844401035 give us π≈3.1326286132812418.
48 edges, side length=0.13080625846028637 give us π≈3.139350203046873.
96 edges, side length=0.0654381656435527 give us π≈3.14103195089053.
192 edges, side length=0.03272346325297234 give us π≈3.1414524722853443.
```

Unser Eigenes Modul

- Die Bedeutung ist ziemlich klar.
- Es gibt ein Modul `my_math` von dem wir die Funktionen `factorial` holen wollen `sqrt`.
- Der Python-Interpreter kennt viele Module.
- Mehrere Module kommen schon mit der Python-Installation mit, wie z. B. `math`.
- Andere werden mit dem Paketmanager pip[13] installiert.

```python
"""Using the mathematics module."""

from my_math import factorial, sqrt  # Import our two functions.

print(f"6!={factorial(6)}")  # Use the `factorial` function.
print(f"\u221A3={sqrt(3.0)}")   # Use the `sqrt` function.
print(f"\u221A(6!*1.5)={sqrt(factorial(6) * 1.5)}")  # Use both.

print("We now use Liu Hui's Method to Approximate \u03c0.")
e: int = 6  # the number of edges: We start with a hexagon, i.e., e=6.
s: float = 1.0  # the side length: Initially 1, i.e., radius is also 1.
for _ in range(6):
    print(f"{e} edges, side length={s} give us \u03c0\u2248{e * s / 2}."
        ↪ )
    e *= 2  # We double the number of edges...
    s = sqrt(2 - sqrt(4 - (s ** 2)))  # ...and recompute the side length
        ↪ .
```

↓ `python3 use_my_math.py` ↓

```
6!=720
√3=1.7320508075688772
√(6!*1.5)=32.863353450309965
We now use Liu Hui's Method to Approximate π.
6 edges, side length=1.0 give us π≈3.0.
12 edges, side length=0.5176380902050417 give us π≈3.10582854123025.
24 edges, side length=0.2610523844401035 give us π≈3.1326286132812418.
48 edges, side length=0.13080625846028637 give us π≈3.139350203046873.
96 edges, side length=0.0654381656435527 give us π≈3.14103195089053.
192 edges, side length=0.03272346325297234 give us π≈3.1414524722853443.
```

Unser Eigenes Modul

- Es gibt ein Modul `my_math` von dem wir die Funktionen `factorial` holen wollen `sqrt`.
- Der Python-Interpreter kennt viele Module.
- Mehrere Module kommen schon mit der Python-Installation mit, wie z. B. `math`.
- Andere werden mit dem Paketmanager pip[13] installiert.
- Das Modul `my_math` wird gefunden, weil wir es in das selbe Verzeichnis tun wie Program `use_my_math.py`.

```python
"""Using the mathematics module."""

from my_math import factorial, sqrt  # Import our two functions.

print(f"6!={factorial(6)}")  # Use the `factorial` function.
print(f"\u221A3={sqrt(3.0)}")   # Use the `sqrt` function.
print(f"\u221A(6!*1.5)={sqrt(factorial(6) * 1.5)}")  # Use both.

print("We now use Liu Hui's Method to Approximate \u03c0.")
e: int = 6  # the number of edges: We start with a hexagon, i.e., e=6.
s: float = 1.0  # the side length: Initially 1, i.e., radius is also 1.
for _ in range(6):
    print(f"{e} edges, side length={s} give us \u03c0\u2248{e * s / 2}.")
    e *= 2  # We double the number of edges...
    s = sqrt(2 - sqrt(4 - (s ** 2)))  # ...and recompute the side length.
```

↓ `python3 use_my_math.py` ↓

```
6!=720
√3=1.7320508075688772
√(6!*1.5)=32.863353450309965
We now use Liu Hui's Method to Approximate π.
6 edges, side length=1.0 give us π≈3.0.
12 edges, side length=0.5176380902050417 give us π≈3.10582854123025.
24 edges, side length=0.2610523844401035 give us π≈3.1326286132812418.
48 edges, side length=0.13080625846028637 give us π≈3.139350203046873.
96 edges, side length=0.0654381656435527 give us π≈3.14103195089053.
192 edges, side length=0.03272346325297234 give us π≈3.1414524722853443.
```

Unser Eigenes Modul

- Der Python-Interpreter kennt viele Module.
- Mehrere Module kommen schon mit der Python-Installation mit, wie z. B. `math`.
- Andere werden mit dem Paketmanager pip[13] installiert.
- Das Modul `my_math` wird gefunden, weil wir es in das selbe Verzeichnis tun wie Program `use_my_math.py`.
- Wir hätten die Datei `my_math.py` auch in ein Unterverzeichnis namens `math_pack` tuen können.

```python
"""Using the mathematics module."""

from my_math import factorial, sqrt  # Import our two functions.

print(f"6!={factorial(6)}")  # Use the `factorial` function.
print(f"\u221A3={sqrt(3.0)}")   # Use the `sqrt` function.
print(f"\u221A(6!*1.5)={sqrt(factorial(6) * 1.5)}")  # Use both.

print("We now use Liu Hui's Method to Approximate \u03c0.")
e: int = 6  # the number of edges: We start with a hexagon, i.e., e=6.
s: float = 1.0  # the side length: Initially 1, i.e., radius is also 1.
for _ in range(6):
    print(f"{e} edges, side length={s} give us \u03c0\u2248{e * s / 2}.")
    e *= 2  # We double the number of edges...
    s = sqrt(2 - sqrt(4 - (s ** 2)))  # ...and recompute the side length.
```

↓ `python3 use_my_math.py` ↓

```
6!=720
√3=1.7320508075688772
√(6!*1.5)=32.863353450309965
We now use Liu Hui's Method to Approximate π.
6 edges, side length=1.0 give us π≈3.0.
12 edges, side length=0.5176380902050417 give us π≈3.10582854123025.
24 edges, side length=0.2610523844401035 give us π≈3.1326286132812418.
48 edges, side length=0.13080625846028637 give us π≈3.139350203046873.
96 edges, side length=0.0654381656435527 give us π≈3.14103195089053.
192 edges, side length=0.03272346325297234 give us π≈3.1414524722853443.
```

Unser Eigenes Modul

- Mehrere Module kommen schon mit der Python-Installation mit, wie z. B. `math`.
- Andere werden mit dem Paketmanager pip[13] installiert.
- Das Modul `my_math` wird gefunden, weil wir es in das selbe Verzeichnis tun wie Program `use_my_math.py`.
- Wir hätten die Datei `my_math.py` auch in ein Unterverzeichnis namens `math_pack` tuen können.
- Dann würden wir unsere Funktionen von `math_pack.my_math` importieren, wobei `math_pack` dann *Paket* genannt werden würde.

```python
"""Using the mathematics module."""

from my_math import factorial, sqrt  # Import our two functions.

print(f"6!={factorial(6)}")  # Use the `factorial` function.
print(f"\u221A3={sqrt(3.0)}")   # Use the `sqrt` function.
print(f"\u221A(6!*1.5)={sqrt(factorial(6) * 1.5)}")  # Use both.

print("We now use Liu Hui's Method to Approximate \u03c0.")
e: int = 6  # the number of edges: We start with a hexagon, i.e., e=6.
s: float = 1.0  # the side length: Initially 1, i.e., radius is also 1.
for _ in range(6):
    print(f"{e} edges, side length={s} give us \u03c0\u2248{e * s / 2}.")
    e *= 2  # We double the number of edges...
    s = sqrt(2 - sqrt(4 - (s ** 2)))  # ...and recompute the side length.
```

↓ `python3 use_my_math.py` ↓

```
6!=720
√3=1.7320508075688772
√(6!*1.5)=32.863353450309965
We now use Liu Hui's Method to Approximate π.
6 edges, side length=1.0 give us π≈3.0.
12 edges, side length=0.5176380902050417 give us π≈3.10582854123025.
24 edges, side length=0.2610523844401035 give us π≈3.1326286132812418.
48 edges, side length=0.13080625846028637 give us π≈3.139350203046873.
96 edges, side length=0.0654381656435527 give us π≈3.14103195089053.
192 edges, side length=0.03272346325297234 give us π≈3.1414524722853443.
```

Unser Eigenes Modul

- Andere werden mit dem Paketmanager pip[13] installiert.
- Das Modul `my_math` wird gefunden, weil wir es in das selbe Verzeichnis tun wie Program `use_my_math.py`.
- Wir hätten die Datei `my_math.py` auch in ein Unterverzeichnis namens `math_pack` tuen können.
- Dann würden wir unsere Funktionen von `math_pack.my_math` importieren, wobei `math_pack` dann *Paket* genannt werden würde.
- Natürlich könnten wir die Verzeichnisse auch tiefer Schachteln.

```python
"""Using the mathematics module."""

from my_math import factorial, sqrt  # Import our two functions.

print(f"6!={factorial(6)}")  # Use the `factorial` function.
print(f"\u221A3={sqrt(3.0)}")   # Use the `sqrt` function.
print(f"\u221A(6!*1.5)={sqrt(factorial(6) * 1.5)}")  # Use both.

print("We now use Liu Hui's Method to Approximate \u03c0.")
e: int = 6  # the number of edges: We start with a hexagon, i.e., e=6.
s: float = 1.0  # the side length: Initially 1, i.e., radius is also 1.
for _ in range(6):
    print(f"{e} edges, side length={s} give us \u03c0\u2248{e * s / 2}.")
    e *= 2  # We double the number of edges...
    s = sqrt(2 - sqrt(4 - (s ** 2)))  # ...and recompute the side length.
```

↓ `python3 use_my_math.py` ↓

```
6!=720
√3=1.7320508075688772
√(6!*1.5)=32.863353450309965
We now use Liu Hui's Method to Approximate π.
6 edges, side length=1.0 give us π≈3.0.
12 edges, side length=0.5176380902050417 give us π≈3.10582854123025.
24 edges, side length=0.2610523844401035 give us π≈3.1326286132812418.
48 edges, side length=0.13080625846028637 give us π≈3.139350203046873.
96 edges, side length=0.0654381656435527 give us π≈3.14103195089053.
192 edges, side length=0.03272346325297234 give us π≈3.1414524722853443.
```

Unser Eigenes Modul

- Das Modul `my_math` wird gefunden, weil wir es in das selbe Verzeichnis tun wie Program `use_my_math.py`.
- Wir hätten die Datei `my_math.py` auch in ein Unterverzeichnis namens `math_pack` tuen können.
- Dann würden wir unsere Funktionen von `math_pack.my_math` importieren, wobei `math_pack` dann *Paket* genannt werden würde.
- Natürlich könnten wir die Verzeichnisse auch tiefer Schachteln.
- Wir könnten Verzeichnis `utils` haben und das Verzeichnis `math_pack` mit unserer Datei `my_math.py` dort hinein tun.

```python
"""Using the mathematics module."""

from my_math import factorial, sqrt  # Import our two functions.

print(f"6!={factorial(6)}")  # Use the `factorial` function.
print(f"\u221A3={sqrt(3.0)}")   # Use the `sqrt` function.
print(f"\u221A(6!*1.5)={sqrt(factorial(6) * 1.5)}")  # Use both.

print("We now use Liu Hui's Method to Approximate \u03c0.")
e: int = 6  # the number of edges: We start with a hexagon, i.e., e=6.
s: float = 1.0  # the side length: Initially 1, i.e., radius is also 1.
for _ in range(6):
    print(f"{e} edges, side length={s} give us \u03c0\u2248{e * s / 2}."
        ↪ )
    e *= 2  # We double the number of edges...
    s = sqrt(2 - sqrt(4 - (s ** 2)))  # ...and recompute the side length
        ↪ .
```

↓ `python3 use_my_math.py` ↓

```
6!=720
√3=1.7320508075688772
√(6!*1.5)=32.863353450309965
We now use Liu Hui's Method to Approximate π.
6 edges, side length=1.0 give us π≈3.0.
12 edges, side length=0.5176380902050417 give us π≈3.10582854123025.
24 edges, side length=0.2610523844401035 give us π≈3.1326286132812418.
48 edges, side length=0.13080625846028637 give us π≈3.139350203046873.
96 edges, side length=0.0654381656435527 give us π≈3.14103195089053.
192 edges, side length=0.03272346325297234 give us π≈3.1414524722853443.
```

Unser Eigenes Modul

- Dann würden wir unsere Funktionen von `math_pack.my_math` importieren, wobei `math_pack` dann *Paket* genannt werden würde.
- Natürlich könnten wir die Verzeichnisse auch tiefer Schachteln.
- Wir könnten Verzeichnis `utils` haben und das Verzeichnis `math_pack` mit unserer Datei `my_math.py` dort hinein tun.
- Dann würden wir unsere Funktionen so importieren: `from utils.math_pack.my_math import`....

```python
"""Using the mathematics module."""

from my_math import factorial, sqrt  # Import our two functions.

print(f"6!={factorial(6)}")  # Use the `factorial` function.
print(f"\u221A3={sqrt(3.0)}")   # Use the `sqrt` function.
print(f"\u221A(6!*1.5)={sqrt(factorial(6) * 1.5)}")  # Use both.

print("We now use Liu Hui's Method to Approximate \u03c0.")
e: int = 6  # the number of edges: We start with a hexagon, i.e., e=6.
s: float = 1.0  # the side length: Initially 1, i.e., radius is also 1.
for _ in range(6):
    print(f"{e} edges, side length={s} give us \u03c0\u2248{e * s / 2}.")
    e *= 2  # We double the number of edges...
    s = sqrt(2 - sqrt(4 - (s ** 2)))  # ...and recompute the side length.
```

↓ `python3 use_my_math.py` ↓

```
6!=720
√3=1.7320508075688772
√(6!*1.5)=32.863353450309965
We now use Liu Hui's Method to Approximate π.
6 edges, side length=1.0 give us π≈3.0.
12 edges, side length=0.5176380902050417 give us π≈3.10582854123025.
24 edges, side length=0.2610523844401035 give us π≈3.1326286132812418.
48 edges, side length=0.13080625846028637 give us π≈3.139350203046873.
96 edges, side length=0.0654381656435527 give us π≈3.14103195089053.
192 edges, side length=0.03272346325297234 give us π≈3.1414524722853443.
```

Unser Eigenes Modul

- Natürlich könnten wir die Verzeichnisse auch tiefer Schachteln.
- Wir könnten Verzeichnis `utils` haben und das Verzeichnis `math_pack` mit unserer Datei `my_math.py` dort hinein tun.
- Dann würden wir unsere Funktionen so importieren: `from utils.math_pack.my_math import`....
- Die Paket- und Modul-Namen werden beim Importieren mit Punkten (.) getrennt.

```python
"""Using the mathematics module."""

from my_math import factorial, sqrt  # Import our two functions.

print(f"6!={factorial(6)}")  # Use the `factorial` function.
print(f"\u221A3={sqrt(3.0)}")   # Use the `sqrt` function.
print(f"\u221A(6!*1.5)={sqrt(factorial(6) * 1.5)}")  # Use both.

print("We now use Liu Hui's Method to Approximate \u03c0.")
e: int = 6  # the number of edges: We start with a hexagon, i.e., e=6.
s: float = 1.0  # the side length: Initially 1, i.e., radius is also 1.
for _ in range(6):
    print(f"{e} edges, side length={s} give us \u03c0\u2248{e * s / 2}.")
    e *= 2  # We double the number of edges...
    s = sqrt(2 - sqrt(4 - (s ** 2)))  # ...and recompute the side length.
```

↓ `python3 use_my_math.py` ↓

```
6!=720
√3=1.7320508075688772
√(6!*1.5)=32.863353450309965
We now use Liu Hui's Method to Approximate π.
6 edges, side length=1.0 give us π≈3.0.
12 edges, side length=0.5176380902050417 give us π≈3.10582854123025.
24 edges, side length=0.2610523844401035 give us π≈3.1326286132812418.
48 edges, side length=0.13080625846028637 give us π≈3.139350203046873.
96 edges, side length=0.0654381656435527 give us π≈3.14103195089053.
192 edges, side length=0.03272346325297234 give us π≈3.1414524722853443.
```

Unser Eigenes Modul

- Wir könnten Verzeichnis `utils` haben und das Verzeichnis `math_pack` mit unserer Datei `my_math.py` dort hinein tun.
- Dann würden wir unsere Funktionen so importieren: `from utils.math_pack.my_math import`....
- Die Paket- und Modul-Namen werden beim Importieren mit Punkten (`.`) getrennt.
- So können wir unser Projekt in Module und Pakete für verschiedene Aufgaben strukturieren.

```python
"""Using the mathematics module."""

from my_math import factorial, sqrt  # Import our two functions.

print(f"6!={factorial(6)}")  # Use the `factorial` function.
print(f"\u221A3={sqrt(3.0)}")   # Use the `sqrt` function.
print(f"\u221A(6!*1.5)={sqrt(factorial(6) * 1.5)}")  # Use both.

print("We now use Liu Hui's Method to Approximate \u03c0.")
e: int = 6  # the number of edges: We start with a hexagon, i.e., e=6.
s: float = 1.0  # the side length: Initially 1, i.e., radius is also 1.
for _ in range(6):
    print(f"{e} edges, side length={s} give us \u03c0\u2248{e * s / 2}.")
    e *= 2  # We double the number of edges...
    s = sqrt(2 - sqrt(4 - (s ** 2)))  # ...and recompute the side length.
```

↓ `python3 use_my_math.py` ↓

```
6!=720
√3=1.7320508075688772
√(6!*1.5)=32.863353450309965
We now use Liu Hui's Method to Approximate π.
6 edges, side length=1.0 give us π≈3.0.
12 edges, side length=0.5176380902050417 give us π≈3.10582854123025.
24 edges, side length=0.2610523844401035 give us π≈3.1326286132812418.
48 edges, side length=0.13080625846028637 give us π≈3.139350203046873.
96 edges, side length=0.0654381656435527 give us π≈3.14103195089053.
192 edges, side length=0.03272346325297234 give us π≈3.1414524722853443.
```

Unser Eigenes Modul

- Dann würden wir unsere Funktionen so importieren: `from utils.math_pack.my_math import`....
- Die Paket- und Modul-Namen werden beim Importieren mit Punkten (`.`) getrennt.
- So können wir unser Projekt in Module und Pakete für verschiedene Aufgaben strukturieren.
- In `use_my_math.py` können wir nun `sqrt` und `factorial` genauso verwenden, als ob wir sie dort drin definiert hätten.

```python
"""Using the mathematics module."""

from my_math import factorial, sqrt  # Import our two functions.

print(f"6!={factorial(6)}")  # Use the `factorial` function.
print(f"\u221A3={sqrt(3.0)}")   # Use the `sqrt` function.
print(f"\u221A(6!*1.5)={sqrt(factorial(6) * 1.5)}")  # Use both.

print("We now use Liu Hui's Method to Approximate \u03c0.")
e: int = 6  # the number of edges: We start with a hexagon, i.e., e=6.
s: float = 1.0  # the side length: Initially 1, i.e., radius is also 1.
for _ in range(6):
    print(f"{e} edges, side length={s} give us \u03c0\u2248{e * s / 2}.")
    e *= 2  # We double the number of edges...
    s = sqrt(2 - sqrt(4 - (s ** 2)))  # ...and recompute the side length.
```

↓ `python3 use_my_math.py` ↓

```
6!=720
√3=1.7320508075688772
√(6!*1.5)=32.863353450309965
We now use Liu Hui's Method to Approximate π.
6 edges, side length=1.0 give us π≈3.0.
12 edges, side length=0.5176380902050417 give us π≈3.10582854123025.
24 edges, side length=0.2610523844401035 give us π≈3.1326286132812418.
48 edges, side length=0.13080625846028637 give us π≈3.139350203046873.
96 edges, side length=0.0654381656435527 give us π≈3.14103195089053.
192 edges, side length=0.03272346325297234 give us π≈3.1414524722853443.
```

Unser Eigenes Modul

- Die Paket- und Modul-Namen werden beim Importieren mit Punkten (`.`) getrennt.
- So können wir unser Projekt in Module und Pakete für verschiedene Aufgaben strukturieren.
- In `use_my_math.py` können wir nun `sqrt` und `factorial` genauso verwenden, als ob wir sie dort drin definiert hätten.
- Wir drucken erstmal ein paar Ergebnisse von `sqrt` und `factorial`.

```python
"""Using the mathematics module."""

from my_math import factorial, sqrt  # Import our two functions.

print(f"6!={factorial(6)}")  # Use the `factorial` function.
print(f"\u221A3={sqrt(3.0)}")   # Use the `sqrt` function.
print(f"\u221A(6!*1.5)={sqrt(factorial(6) * 1.5)}")  # Use both.

print("We now use Liu Hui's Method to Approximate \u03c0.")
e: int = 6  # the number of edges: We start with a hexagon, i.e., e=6.
s: float = 1.0  # the side length: Initially 1, i.e., radius is also 1.
for _ in range(6):
    print(f"{e} edges, side length={s} give us \u03c0\u2248{e * s / 2}.")
    e *= 2  # We double the number of edges...
    s = sqrt(2 - sqrt(4 - (s ** 2)))  # ...and recompute the side length.
```

↓ `python3 use_my_math.py` ↓

```
6!=720
√3=1.7320508075688772
√(6!*1.5)=32.863353450309965
We now use Liu Hui's Method to Approximate π.
6 edges, side length=1.0 give us π≈3.0.
12 edges, side length=0.5176380902050417 give us π≈3.10582854123025.
24 edges, side length=0.2610523844401035 give us π≈3.1326286132812418.
48 edges, side length=0.13080625846028637 give us π≈3.139350203046873.
96 edges, side length=0.0654381656435527 give us π≈3.14103195089053.
192 edges, side length=0.03272346325297234 give us π≈3.1414524722853443.
```

Unser Eigenes Modul

- So können wir unser Projekt in Module und Pakete für verschiedene Aufgaben strukturieren.
- In `use_my_math.py` können wir nun `sqrt` und `factorial` genauso verwenden, als ob wir sie dort drin definiert hätten.
- Wir drucken erstmal ein paar Ergebnisse von `sqrt` und `factorial`.
- Wir kopieren auch etwas Kode von einem alten Beispiel, wo wir LIU Hui (刘徽) seine Methode zum Annähern von π verwendet haben.

```python
"""Using the mathematics module."""

from my_math import factorial, sqrt  # Import our two functions.

print(f"6!={factorial(6)}")  # Use the `factorial` function.
print(f"\u221A3={sqrt(3.0)}")   # Use the `sqrt` function.
print(f"\u221A(6!*1.5)={sqrt(factorial(6) * 1.5)}")  # Use both.

print("We now use Liu Hui's Method to Approximate \u03c0.")
e: int = 6  # the number of edges: We start with a hexagon, i.e., e=6.
s: float = 1.0  # the side length: Initially 1, i.e., radius is also 1.
for _ in range(6):
    print(f"{e} edges, side length={s} give us \u03c0\u2248{e * s / 2}.")
    e *= 2  # We double the number of edges...
    s = sqrt(2 - sqrt(4 - (s ** 2)))  # ...and recompute the side length.
```

↓ `python3 use_my_math.py` ↓

```
6!=720
√3=1.7320508075688772
√(6!*1.5)=32.863353450309965
We now use Liu Hui's Method to Approximate π.
6 edges, side length=1.0 give us π≈3.0.
12 edges, side length=0.5176380902050417 give us π≈3.10582854123025.
24 edges, side length=0.2610523844401035 give us π≈3.1326286132812418.
48 edges, side length=0.13080625846028637 give us π≈3.139350203046873.
96 edges, side length=0.0654381656435527 give us π≈3.14103195089053.
192 edges, side length=0.03272346325297234 give us π≈3.1414524722853443.
```

Unser Eigenes Modul

- In `use_my_math.py` können wir nun `sqrt` und `factorial` genauso verwenden, als ob wir sie dort drin definiert hätten.
- Wir drucken erstmal ein paar Ergebnisse von `sqrt` und `factorial`.
- Wir kopieren auch etwas Kode von einem alten Beispiel, wo wir LIU Hui (刘徽) seine Methode zum Annähern von π verwendet haben.
- Diesmal nehmen wir unsere eigene Methode zum Berechnen der Quadratwurzel und nicht die Funktion aus dem `math`-Modul.

```python
"""Using the mathematics module."""

from my_math import factorial, sqrt  # Import our two functions.

print(f"6!={factorial(6)}")  # Use the `factorial` function.
print(f"\u221A3={sqrt(3.0)}")   # Use the `sqrt` function.
print(f"\u221A(6!*1.5)={sqrt(factorial(6) * 1.5)}")  # Use both.

print("We now use Liu Hui's Method to Approximate \u03c0.")
e: int = 6  # the number of edges: We start with a hexagon, i.e., e=6.
s: float = 1.0  # the side length: Initially 1, i.e., radius is also 1.
for _ in range(6):
    print(f"{e} edges, side length={s} give us \u03c0\u2248{e * s / 2}."
        ↪ )
    e *= 2  # We double the number of edges...
    s = sqrt(2 - sqrt(4 - (s ** 2)))  # ...and recompute the side length
        ↪ .
```

↓ `python3 use_my_math.py` ↓

```
6!=720
√3=1.7320508075688772
√(6!*1.5)=32.863353450309965
We now use Liu Hui's Method to Approximate π.
6 edges, side length=1.0 give us π≈3.0.
12 edges, side length=0.5176380902050417 give us π≈3.10582854123025.
24 edges, side length=0.2610523844401035 give us π≈3.1326286132812418.
48 edges, side length=0.13080625846028637 give us π≈3.139350203046873.
96 edges, side length=0.0654381656435527 give us π≈3.14103195089053.
192 edges, side length=0.03272346325297234 give us π≈3.1414524722853443.
```

Unser Eigenes Modul

- Wir drucken erstmal ein paar Ergebnisse von `sqrt` und `factorial`.
- Wir kopieren auch etwas Kode von einem alten Beispiel, wo wir LIU Hui (刘徽) seine Methode zum Annähern von π verwendet haben.
- Diesmal nehmen wir unsere eigene Methode zum Berechnen der Quadratwurzel und nicht die Funktion aus dem `math`-Modul.
- Interessanterweise haben die letzten beiden Annäherungsschritte die selben Ergebnisse wie damals.

```python
"""Using the mathematics module."""

from my_math import factorial, sqrt  # Import our two functions.

print(f"6!={factorial(6)}")  # Use the `factorial` function.
print(f"\u221A3={sqrt(3.0)}")   # Use the `sqrt` function.
print(f"\u221A(6!*1.5)={sqrt(factorial(6) * 1.5)}")  # Use both.

print("We now use Liu Hui's Method to Approximate \u03c0.")
e: int = 6  # the number of edges: We start with a hexagon, i.e., e=6.
s: float = 1.0  # the side length: Initially 1, i.e., radius is also 1.
for _ in range(6):
    print(f"{e} edges, side length={s} give us \u03c0\u2248{e * s / 2}.")
    e *= 2  # We double the number of edges...
    s = sqrt(2 - sqrt(4 - (s ** 2)))  # ...and recompute the side length.
```

↓ `python3 use_my_math.py` ↓

```
6!=720
√3=1.7320508075688772
√(6!*1.5)=32.863353450309965
We now use Liu Hui's Method to Approximate π.
6 edges, side length=1.0 give us π≈3.0.
12 edges, side length=0.5176380902050417 give us π≈3.10582854123025.
24 edges, side length=0.2610523844401035 give us π≈3.1326286132812418.
48 edges, side length=0.13080625846028637 give us π≈3.139350203046873.
96 edges, side length=0.0654381656435527 give us π≈3.14103195089053.
192 edges, side length=0.03272346325297234 give us π≈3.1414524722853443.
```

Zusammenfassung

Zusammenfassung

- Seit wir `while`-Schleifen benutzen können, waren wir in der Lage, im Grunde jede durch einen Computer berechenbare Funktion zu implementieren.

Zusammenfassung

- Seit wir `while`-Schleifen benutzen können, waren wir in der Lage, im Grunde jede durch einen Computer berechenbare Funktion zu implementieren.
- Wir waren aber auf der technischen Ebene eingeschränkt.

Zusammenfassung

- Seit wir `while`-Schleifen benutzen können, waren wir in der Lage, im Grunde jede durch einen Computer berechenbare Funktion zu implementieren.
- Wir waren aber auf der technischen Ebene eingeschränkt.
- Wir könnten zwar, theoretisch, beliebig komplexe Programme implementieren.

Zusammenfassung

- Seit wir `while`-Schleifen benutzen können, waren wir in der Lage, im Grunde jede durch einen Computer berechenbare Funktion zu implementieren.
- Wir waren aber auf der technischen Ebene eingeschränkt.
- Wir könnten zwar, theoretisch, beliebig komplexe Programme implementieren.
- Ohne Funktionen hätte das aber sicherlich zu sich oftmals wiederholenden Kode geführt.

Zusammenfassung

- Seit wir `while`-Schleifen benutzen können, waren wir in der Lage, im Grunde jede durch einen Computer berechenbare Funktion zu implementieren.
- Wir waren aber auf der technischen Ebene eingeschränkt.
- Wir könnten zwar, theoretisch, beliebig komplexe Programme implementieren.
- Ohne Funktionen hätte das aber sicherlich zu sich oftmals wiederholenden Kode geführt.
- Und ohne Module hätten wir irgendwann eine riesige, nicht wartbare und unleserliche Datei gehabt.

Zusammenfassung

- Seit wir `while`-Schleifen benutzen können, waren wir in der Lage, im Grunde jede durch einen Computer berechenbare Funktion zu implementieren.
- Wir waren aber auf der technischen Ebene eingeschränkt.
- Wir könnten zwar, theoretisch, beliebig komplexe Programme implementieren.
- Ohne Funktionen hätte das aber sicherlich zu sich oftmals wiederholenden Kode geführt.
- Und ohne Module hätten wir irgendwann eine riesige, nicht wartbare und unleserliche Datei gehabt.
- Diese Beschränkungen haben wir jetzt durchbrochen.

Zusammenfassung

- Seit wir `while`-Schleifen benutzen können, waren wir in der Lage, im Grunde jede durch einen Computer berechenbare Funktion zu implementieren.
- Wir waren aber auf der technischen Ebene eingeschränkt.
- Wir könnten zwar, theoretisch, beliebig komplexe Programme implementieren.
- Ohne Funktionen hätte das aber sicherlich zu sich oftmals wiederholenden Kode geführt.
- Und ohne Module hätten wir irgendwann eine riesige, nicht wartbare und unleserliche Datei gehabt.
- Diese Beschränkungen haben wir jetzt durchbrochen.
- Ab jetzt können wir beliebig große und beliebig komplexe Applikationen mit Python programmieren.

Zusammenfassung

- Seit wir `while`-Schleifen benutzen können, waren wir in der Lage, im Grunde jede durch einen Computer berechenbare Funktion zu implementieren.
- Wir waren aber auf der technischen Ebene eingeschränkt.
- Wir könnten zwar, theoretisch, beliebig komplexe Programme implementieren.
- Ohne Funktionen hätte das aber sicherlich zu sich oftmals wiederholenden Kode geführt.
- Und ohne Module hätten wir irgendwann eine riesige, nicht wartbare und unleserliche Datei gehabt.
- Diese Beschränkungen haben wir jetzt durchbrochen.
- Ab jetzt können wir beliebig große und beliebig komplexe Applikationen mit Python programmieren.
- Wir können verschiedene Funktionen für verschiedene Teilaufgaben implementieren.

Zusammenfassung

- Seit wir `while`-Schleifen benutzen können, waren wir in der Lage, im Grunde jede durch einen Computer berechenbare Funktion zu implementieren.
- Wir waren aber auf der technischen Ebene eingeschränkt.
- Wir könnten zwar, theoretisch, beliebig komplexe Programme implementieren.
- Ohne Funktionen hätte das aber sicherlich zu sich oftmals wiederholenden Kode geführt.
- Und ohne Module hätten wir irgendwann eine riesige, nicht wartbare und unleserliche Datei gehabt.
- Diese Beschränkungen haben wir jetzt durchbrochen.
- Ab jetzt können wir beliebig große und beliebig komplexe Applikationen mit Python programmieren.
- Wir können verschiedene Funktionen für verschiedene Teilaufgaben implementieren.
- Wir können Funktionen für ähnliche Themengebiete in verschiedene Module gruppieren.

Zusammenfassung

- Seit wir `while`-Schleifen benutzen können, waren wir in der Lage, im Grunde jede durch einen Computer berechenbare Funktion zu implementieren.
- Wir waren aber auf der technischen Ebene eingeschränkt.
- Wir könnten zwar, theoretisch, beliebig komplexe Programme implementieren.
- Ohne Funktionen hätte das aber sicherlich zu sich oftmals wiederholenden Kode geführt.
- Und ohne Module hätten wir irgendwann eine riesige, nicht wartbare und unleserliche Datei gehabt.
- Diese Beschränkungen haben wir jetzt durchbrochen.
- Ab jetzt können wir beliebig große und beliebig komplexe Applikationen mit Python programmieren.
- Wir können verschiedene Funktionen für verschiedene Teilaufgaben implementieren.
- Wir können Funktionen für ähnliche Themengebiete in verschiedene Module gruppieren.
- Wir können Module sogar hierarchisch in Pakete gruppieren.

Zusammenfassung

- Seit wir `while`-Schleifen benutzen können, waren wir in der Lage, im Grunde jede durch einen Computer berechenbare Funktion zu implementieren.
- Wir waren aber auf der technischen Ebene eingeschränkt.
- Wir könnten zwar, theoretisch, beliebig komplexe Programme implementieren.
- Ohne Funktionen hätte das aber sicherlich zu sich oftmals wiederholenden Kode geführt.
- Und ohne Module hätten wir irgendwann eine riesige, nicht wartbare und unleserliche Datei gehabt.
- Diese Beschränkungen haben wir jetzt durchbrochen.
- Ab jetzt können wir beliebig große und beliebig komplexe Applikationen mit Python programmieren.
- Wir können verschiedene Funktionen für verschiedene Teilaufgaben implementieren.
- Wir können Funktionen für ähnliche Themengebiete in verschiedene Module gruppieren.
- Wir können Module sogar hierarchisch in Pakete gruppieren.
- Damit können wir wohlstrukturierte und wartbare Software entwickeln.

谢谢您门！

Thank you!

Vielen Dank!

References I

[1] Daniel J. Barrett. *Efficient Linux at the Command Line*. Sebastopol, CA, USA: O'Reilly Media, Inc., Feb. 2022. ISBN: 978-1-0981-1340-7 (siehe S. 81, 82).

[2] Ethan Bommarito und Michael Bommarito. *An Empirical Analysis of the Python Package Index (PyPI)*. arXiv.org: Computing Research Repository (CoRR) abs/1907.11073. Ithaca, NY, USA: Cornell Universiy Library, 26. Juli 2019. doi:10.48550/arXiv.1907.11073. URL: https://arxiv.org/abs/1907.11073 (besucht am 2024-08-17). arXiv:1907.11073v2 [cs.SE] 26 Jul 2019 (siehe S. 82).

[3] Ed Bott. *Windows 11 Inside Out*. Hoboken, NJ, USA: Microsoft Press, Pearson Education, Inc., Feb. 2023. ISBN: 978-0-13-769132-6 (siehe S. 81).

[4] Ron Brash und Ganesh Naik. *Bash Cookbook*. Birmingham, England, UK: Packt Publishing Ltd, Juli 2018. ISBN: 978-1-78862-936-2 (siehe S. 81).

[5] David Clinton und Christopher Negus. *Ubuntu Linux Bible*. 10. Aufl. Bible Series. Chichester, West Sussex, England, UK: John Wiley and Sons Ltd., 10. Nov. 2020. ISBN: 978-1-119-72233-5 (siehe S. 82).

[6] Leonhard Euler. "An Essay on Continued Fractions". Übers. von Myra F. Wyman und Bostwick F. Wyman. *Mathematical Systems Theory* 18(1):295–328, Dez. 1985. New York, NY, USA: Springer Science+Business Media, LLC. ISSN: 1432-4350. doi:10.1007/BF01699475. URL: https://www.researchgate.net/publication/301720080 (besucht am 2024-09-24). Translation of[7]. (Siehe S. 77).

[7] Leonhard Euler. "De Fractionibus Continuis Dissertation". *Commentarii Academiae Scientiarum Petropolitanae* 9:98–137, 1737–1744. Petropolis (St. Petersburg), Russia: Typis Academiae. URL: https://scholarlycommons.pacific.edu/cgi/viewcontent.cgi?article=1070 (besucht am 2024-09-24). See[6] for a translation. (Siehe S. 77, 83).

[8] Michael Filaseta. "The Transcendence of e and π". In: *Math 785: Transcendental Number Theory*. Columbia, SC, USA: University of South Carolina, Frühling 2011. Kap. 6. URL: https://people.math.sc.edu/filaseta/gradcourses/Math785/Math785Notes6.pdf (besucht am 2024-07-05) (siehe S. 83).

[9] David Goodger und Guido van Rossum. *Docstring Conventions*. Python Enhancement Proposal (PEP) 257. Beaverton, OR, USA: Python Software Foundation (PSF), 29. Mai–13. Juni 2001. URL: https://peps.python.org/pep-0257 (besucht am 2024-07-27) (siehe S. 81).

References II

[10] Michael Hausenblas. *Learning Modern Linux*. Sebastopol, CA, USA: O'Reilly Media, Inc., Apr. 2022. ISBN: 978-1-0981-0894-6 (siehe S. 81).

[11] Matthew Helmke. *Ubuntu Linux Unleashed 2021 Edition*. 14. Aufl. Reading, MA, USA: Addison-Wesley Professional, Aug. 2020. ISBN: 978-0-13-668539-5 (siehe S. 82).

[12] John Hunt. *A Beginners Guide to Python 3 Programming*. 2. Aufl. Undergraduate Topics in Computer Science (UTICS). Cham, Switzerland: Springer, 2023. ISBN: 978-3-031-35121-1. doi:10.1007/978-3-031-35122-8 (siehe S. 82).

[13] *Python 3 Documentation*. *Installing Python Modules*. Beaverton, OR, USA: Python Software Foundation (PSF), 2001–2025. URL: https://docs.python.org/3/installing (besucht am 2024-08-17) (siehe S. 31–54, 81).

[14] Arthur Jones, Kenneth R. Pearson und Sidney A. Morris. "Transcendence of e and π". In: *Abstract Algebra and Famous Impossibilities*. Universitext (UTX). New York, NY, USA: Springer New York, 1991. Kap. 9, S. 115–161. ISSN: 0172-5939. ISBN: 978-1-4419-8552-1. doi:10.1007/978-1-4419-8552-1_8 (siehe S. 83).

[15] Łukasz Langa. *Literature Overview for Type Hints*. Python Enhancement Proposal (PEP) 482. Beaverton, OR, USA: Python Software Foundation (PSF), 8. Jan. 2015. URL: https://peps.python.org/pep-0482 (besucht am 2024-10-09) (siehe S. 82).

[16] Kent D. Lee und Steve Hubbard. *Data Structures and Algorithms with Python*. Undergraduate Topics in Computer Science (UTICS). Cham, Switzerland: Springer, 2015. ISBN: 978-3-319-13071-2. doi:10.1007/978-3-319-13072-9 (siehe S. 82).

[17] Jukka Lehtosalo, Ivan Levkivskyi, Jared Hance, Ethan Smith, Guido van Rossum, Jelle "JelleZijlstra" Zijlstra, Michael J. Sullivan, Shantanu Jain, Xuanda Yang, Jingchen Ye, Nikita Sobolev und Mypy Contributors. *Mypy – Static Typing for Python*. San Francisco, CA, USA: GitHub Inc, 2024. URL: https://github.com/python/mypy (besucht am 2024-08-17) (siehe S. 81).

[18] Mark Lutz. *Learning Python*. 6. Aufl. Sebastopol, CA, USA: O'Reilly Media, Inc., März 2025. ISBN: 978-1-0981-7130-8 (siehe S. 82).

[19] Cameron Newham und Bill Rosenblatt. *Learning the Bash Shell – Unix Shell Programming: Covers Bash 3.0*. 3. Aufl. Sebastopol, CA, USA: O'Reilly Media, Inc., 2005. ISBN: 978-0-596-00965-6 (siehe S. 81).

[20] Ivan Niven. "The Transcendence of π". *The American Mathematical Monthly* 46(8):469–471, Okt. 1939. London, England, UK: Taylor and Francis Ltd. ISSN: 1930-0972. doi:10.2307/2302515 (siehe S. 83).

References III

[21] Yasset Pérez-Riverol, Laurent Gatto, Rui Wang, Timo Sachsenberg, Julian Uszkoreit, Felipe da Veiga Leprevost, Christian Fufezan, Tobias Ternent, Stephen J. Eglen, Daniel S. Katz, Tom J. Pollard, Alexander Konovalov, Robert M. Flight, Kai Blin und Juan Antonio Vizcaíno. "Ten Simple Rules for Taking Advantage of Git and GitHub". *PLOS Computational Biology* 12(7), 14. Juli 2016. San Francisco, CA, USA: Public Library of Science (PLOS). ISSN: 1553-7358. doi:10.1371/JOURNAL.PCBI.1004947 (siehe S. 81).

[22] pip Developers. *pip Documentation v24.3.1*. Beaverton, OR, USA: Python Software Foundation (PSF), 27. Okt. 2024. URL: https://pip.pypa.io (besucht am 2024-12-25) (siehe S. 81).

[23] Ellen Siever, Stephen Figgins, Robert Love und Arnold Robbins. *Linux in a Nutshell*. 6. Aufl. Sebastopol, CA, USA: O'Reilly Media, Inc., Sep. 2009. ISBN: 978-0-596-15448-6 (siehe S. 81).

[24] Anna Skoulikari. *Learning Git*. Sebastopol, CA, USA: O'Reilly Media, Inc., Mai 2023. ISBN: 978-1-0981-3391-7 (siehe S. 81).

[25] "The Import System". In: *Python 3 Documentation. The Python Language Reference*. Beaverton, OR, USA: Python Software Foundation (PSF), 2001–2025. Kap. 5. URL: https://docs.python.org/3/reference/import.html (besucht am 2024-10-01) (siehe S. 16–23).

[26] *The Python Package Index (PyPI)*. Beaverton, OR, USA: Python Software Foundation (PSF), 2024. URL: https://pypi.org (besucht am 2024-08-17) (siehe S. 82).

[27] Linus Torvalds. "The Linux Edge". *Communications of the ACM (CACM)* 42(4):38–39, Apr. 1999. New York, NY, USA: Association for Computing Machinery (ACM). ISSN: 0001-0782. doi:10.1145/299157.299165 (siehe S. 81).

[28] Mariot Tsitoara. *Beginning Git and GitHub: Version Control, Project Management and Teamwork for the New Developer*. New York, NY, USA: Apress Media, LLC, März 2024. ISBN: 979-8-8688-0215-7 (siehe S. 81, 82).

[29] Marat Valiev, Bogdan Vasilescu und James D. Herbsleb. "Ecosystem-Level Determinants of Sustained Activity in Open-Source Projects: A Case Study of the PyPI Ecosystem". In: *ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/SIGSOFT FSE'2018)*. 4.–9. Nov. 2018, Lake Buena Vista, FL, USA. Hrsg. von Gary T. Leavens, Alessandro F. Garcia und Corina S. Păsăreanu. New York, NY, USA: Association for Computing Machinery (ACM), 2018, S. 644–655. ISBN: 978-1-4503-5573-5. doi:10.1145/3236024.3236062 (siehe S. 82).

References IV

[30] Guido van Rossum und Łukasz Langa. *Type Hints*. Python Enhancement Proposal (PEP) 484. Beaverton, OR, USA: Python Software Foundation (PSF), 29. Sep. 2014. URL: https://peps.python.org/pep-0484 (besucht am 2024-08-22) (siehe S. 82).

[31] Guido van Rossum, Barry Warsaw und Alyssa Coghlan. *Style Guide for Python Code*. Python Enhancement Proposal (PEP) 8. Beaverton, OR, USA: Python Software Foundation (PSF), 5. Juli 2001. URL: https://peps.python.org/pep-0008 (besucht am 2024-07-27) (siehe S. 30, 81).

[32] Sander van Vugt. *Linux Fundamentals*. 2. Aufl. Hoboken, NJ, USA: Pearson IT Certification, Juni 2022. ISBN: 978-0-13-792931-3 (siehe S. 81).

[33] Thomas Weise (汤卫思). *Programming with Python*. Hefei, Anhui, China (中国安徽省合肥市): Hefei University (合肥大学), School of Artificial Intelligence and Big Data (人工智能与大数据学院), Institute of Applied Optimization (应用优化研究所, IAO), 2024–2025. URL: https://thomasweise.github.io/programmingWithPython (besucht am 2025-01-05) (siehe S. 81, 82).

[34] Giorgio Zarrelli. *Mastering Bash*. Birmingham, England, UK: Packt Publishing Ltd, Juni 2017. ISBN: 978-1-78439-687-9 (siehe S. 81).

Glossary (in English) I

Bash is a the shell used under Ubuntu Linux, i.e., the program that "runs" in the terminal and interprets your commands, allowing you to start and interact with other programs[4,19,34]. Learn more at https://www.gnu.org/software/bash.

docstring Docstrings are special string constants in Python that contain documentation for modules or functions[9]. They must be delimited by `"""..."""`[9,31].

Git is a distributed Version Control Systems (VCS) which allows multiple users to work on the same code while preserving the history of the code changes[24,28]. Learn more at https://git-scm.com.

GitHub is a website where software projects can be hosted and managed via the Git VCS[21,28]. Learn more at https://github.com.

IT information technology

Linux is the leading open source operating system, i.e., a free alternative for Microsoft Windows[1,10,23,27,32]. We recommend using it for this course, for software development, and for research. Learn more at https://www.linux.org. Its variant Ubuntu is particularly easy to use and install.

Microsoft Windows is a commercial proprietary operating system[3]. It is widely spread, but we recommend using a Linux variant such as Ubuntu for software development and for our course. Learn more at https://www.microsoft.com/windows.

Mypy is a static type checking tool for Python[17] that makes use of type hints. Learn more at https://github.com/python/mypy and in[33].

pip is the standard tool to install Python software packages from the PyPI repository[13,22]. To install a package `thepackage` hosted on PyPI, type `pip install thepackage` into the terminal. Learn more at https://packaging.python.org/installing.

Glossary (in English) II

PyPI The Python Package Index (PyPI) is an online repository that provides the software packages that you can install with pip[2,26,29]. Learn more at https://pypi.org.

Python The Python programming language[12,16,18,33], i.e., what you will learn about in our book[33]. Learn more at https://python.org.

terminal A terminal is a text-based window where you can enter commands and execute them[1,5]. Knowing what a terminal is and how to use it is very essential in any programming- or system administration-related task. If you want to open a terminal under Microsoft Windows, you can Druck auf [⊞]+[R], dann Schreiben von `cmd`, dann Druck auf [↵]. Under Ubuntu Linux, [Ctrl]+[Alt]+[T] opens a terminal, which then runs a Bash shell inside.

type hint are annotations that help programmers and static code analysis tools such as Mypy to better understand what type a variable or function parameter is supposed to be[15,30]. Python is a dynamically typed programming language where you do not need to specify the type of, e.g., a variable. This creates problems for code analysis, both automated as well as manual: For example, it may not always be clear whether a variable or function parameter should be an integer or floating point number. The annotations allow us to explicitly state which type is expected. They are *ignored* during the program execution. They are a basically a piece of documentation.

Ubuntu is a variant of the open source operating system Linux[5,11]. We recommend that you use this operating system to follow this class, for software development, and for research. Learn more at https://ubuntu.com. If you are in China, you can download it from https://mirrors.ustc.edu.cn/ubuntu-releases.

VCS A *Version Control System* is a software which allows you to manage and preserve the historical development of your program code[28]. A distributed VCS allows multiple users to work on the same code and upload their changes to the server, which then preserves the change history. The most popular distributed VCS is Git.

Glossary (in English) III

π is the ratio of the circumference U of a circle and its diameter d, i.e., $\pi = U/d$. $\pi \in \mathbb{R}$ is an irrational and transcendental number[8,14,20], which is approximately $\pi \approx 3.141\,592\,653\,589\,793\,238\,462\,643$. In Python, it is provided by the `math` module as constant `pi` with value `3.141592653589793`.

e is Euler's number[7], the base of the natural logarithm. $e \in \mathbb{R}$ is an irrational and transcendental number[8,14], which is approximately $e \approx 2.718\,281\,828\,459\,045\,235\,360$. In Python, it is provided by the `math` module as constant `e` with value `2.718281828459045`.

$\mathbb{R}$ the set of the real numbers.